Supplier Payment Control

Technical guide (technical overview)

Technical name	`odt_supplier_payment_control`
Module version	`19.0.1.0.0`
Odoo version	19.0
Document type	Technical guide (technical overview)
Language	English
Generated on	2026-10-15

Contents

Overview 2
- What the module does 2
- Design 2

Dependencies 4
- Odoo modules 4
- External dependencies 4

Prerequisites 5
- Installation order 5
- Required configuration 5
- Required data 5
- Access rights 5

Quality 7
- Tests 7
- Licence and authorship 7
- Compliance with the OdooTec module standard 8

Open points 9
- Unfinished work 9
- Translations 9
- Reports 10
- Documentation gaps 10

Overview

What the module does

The module controls outgoing vendor payments. A vendor bill can be paid only when its three-way match (Should Be Paid, field `release_to_pay` of `account_3way_match`) is Yes, unless it is a service bill raised without a purchase order. A vendor payment above the company thresholds needs one approval per level (Finance Manager, Operations Manager, Chief Executive) before it can be posted. A payment that repeats an earlier one (same vendor, amount and memo) is blocked until an administrator releases it. Posting is also refused for an untrusted recipient bank account and for a shareholder payment whose destination account is marked payroll.

Technically, every check runs in overrides of standard entry points (`action_post`, `_generate_journal_entry`, `action_create_payments`, `validate_batch`), plus one new model, `payment.approval`, that stores one row per approval given. Only payments with `payment_type = 'outbound'` and `partner_type = 'supplier'` are controlled (`models/account_payment.py:299`); customer and incoming payments behave as standard.

Design

Model	New / inherited	Role
`payment.approval`	New	One approval of one payment at one level, with approver and date. Read-only after creation: no group has write or unlink access (`security/ir.model.access.csv`).
`account.payment`	Inherited	`approval_state` (stored compute), `approval_ids`, `duplicate_of_id` (stored compute), `is_duplicate_released`; buttons `action_approve`, `action_release_duplicate`; overrides `action_post` and `_generate_journal_entry`; constraint `_check_shareholder_account`.
`account.payment.register`	Inherited (wizard)	`action_create_payments` refuses bills not cleared to pay, then calls standard with context `defer_payment_approval`.
`account.batch.payment`	Inherited	`validate_batch` refuses an outbound batch holding a bill not cleared to pay.
`account.move`	Inherited	`_is_payment_blocked` and `_is_service_bill_without_order` (no new field).
`res.company`	Inherited	`approval_threshold_1/2/3` (Monetary) with an ordering constraint.
`res.partner`	Inherited	`is_shareholder`.
`account.account`	Inherited	`is_payroll`.

Key flows

- **Approval levels.** `LEVEL_RULES` (`models/account_payment.py:23`) maps each level to its company threshold field and its approver group: Finance Manager = `account.group_account_manager`, Operations Manager = `group_ops_approver`, Chief Executive = `group_ceo_approver`. A level is required when its threshold is set and the amount, converted to company currency at the payment date, is strictly above it (`_get_required_approval_levels`, line 316). Levels are approved strictly in order; `action_approve` gives only the next missing level and checks the group with `has_group` (line 206).
- **Approval status.** `approval_state` is a stored compute over state, amount, currency, date, company, approvals and duplicate fields (line 105). It is empty for payments the control does not cover, so the form badge, the Approvals tab and the list filters only appear on vendor payments.
- **Deferred posting from bills.** The Register Payment wizard sets `defer_payment_approval`. Under that context, `action_post` still raises for hard errors (untrusted bank account, shareholder on payroll) but silently leaves a payment that waits for approval or a duplicate release in draft (line 152). Posting the payment later through `action_post` without the context runs all checks.
- **Journal entry generated at creation.** When the payment method has an outstanding account, Odoo 19 generates the payment's entry inside `account.payment.create` and writes `state = 'in_process'` without calling `action_post` (core `addons/account/models/account_payment.py:926` and `:1073`). The override of `_generate_journal_entry` (line 176) puts a waiting vendor payment back to draft and leaves its entry unposted; `action_post` posts it once the controls pass. See Quality for the history of this fix.
- **Duplicate detection.** `_find_duplicate` (line 354) searches, with `sudo`, the earliest earlier outbound supplier payment of the same company with the same partner, amount and non-empty memo, excluding cancelled and rejected ones. Only drafts are recomputed, so a posted payment keeps its link. The release writes `is_duplicate_released` and posts a note on both payments; the earlier payment is not reset.
- **Shareholder rule.** Checked twice: as a posting error in `_get_payment_control_error` and as an `@api.constrains` on state, destination account and partner (line 138), so it also holds for payments that reach In Process by another path.

Notable decisions

- Approvals are rows of a separate model, not booleans on the payment. This keeps who and when per level and gives an audit list (**Accounting › Vendors › Payment Approvals**).
- `sudo` is used in three narrow places, each commented in the code: reading approvals for users without access (line 335), posting chatter notes for approvers who hold read-only accounting rights, and the duplicate search across the whole company.
- No `post_init_hook` or `uninstall_hook`. No cron, controller or report.

Dependencies

Odoo modules

Module	Why it is needed (as the code shows)
`account_accountant`	Accounting base (Enterprise); payments and bills.
`account_batch_payment`	`account.batch.payment.validate_batch` is overridden.
`purchase`	`purchase_line_id` on bill lines, used to exempt service bills without a purchase order.
`account_3way_match`	`release_to_pay` on vendor bills (Should Be Paid), the basis of the payment block.

`stock` is not declared but is needed in practice for receipts to drive the three-way match; it is installed on the documentation database. This is an observation, not a defect: a bill can still be matched against ordered quantities for products billed on ordered quantities.

External dependencies

None declared. No third-party Python import was found in the module code. The tests import `freezegun`, which ships with Odoo's test requirements.

Prerequisites

Installation order

Install the module like any other; Odoo installs the four dependencies first. It does not depend on the sibling RVK modules (`odt_customer_credit_control`, `odt_quotation_payment_terms`).

Required configuration

- **Thresholds per company.** **Settings › Companies › (company) › Payment Approval**: `approval_threshold_1/2/3`. All three default to 0, and 0 means "level not used": until they are set, no vendor payment needs approval. A set threshold must be above the set threshold of the level before it, and none can be negative (`models/res_company.py:42`).
- **Trusted bank accounts.** A vendor payment with a recipient bank account whose standard Trusted flag (`allow_out_payment`) is off is refused.
- **Payroll accounts and shareholders.** Mark payroll expense accounts with **Payroll Expense Account** (`is_payroll`) and shareholder contacts with **Shareholder** (`is_shareholder`).

Required data

No data file is loaded apart from security. The three-way match needs purchase orders, receipts and bills linked to purchase order lines.

Access rights

Group	Can do
`account.group_account_manager` (Accounting / Administrator) = Finance Manager	Approve the first level; read and create `payment.approval`.
`group_ops_approver` (Vendor Payment Approval / Payment approver (Operations))	Approve the second level. Implies `account.group_account_readonly`.
`group_ceo_approver` (Vendor Payment Approval / Payment approver (Chief Executive))	Approve the third level. Implies `account.group_account_readonly`.
`account.group_account_user`	Read approvals; sees the Payment Approvals menu, the Approvals tab and the destination account on payments.
`base.group_system` (Settings)	Release a blocked duplicate.

Record rule `payment_approval_rule_company` limits approvals to the user's companies.

> **Note:** `account.group_account_user` is labelled Bookkeeper and becomes selectable on the user form, and implied by Accounting / Administrator, only when the Enterprise **Accounting** app (`accountant`) is installed (`Enterprise/accountant/security/accounting_security.xml`).

> The module does not depend on `accountant`. On the documentation database `accountant` is not installed: a user with only Accounting / Administrator was checked on 2026-10-15 and does not see the Payment Approvals menu, the Approvals tab or the destination account, although the **Approve** button is visible to them. Install the Accounting app, or grant the group by other means, before go-live.

Quality

Tests

28 test methods in 3 files, all imported by `tests/__init__.py`, all tagged `post_install`, `-at_install`, `odt_supplier_payment_control`:

File	Methods	Covers
`tests/test_bill_matching.py`	4	Unmatched bill refused, corrected bill paid, batch naming the bill, service bill exempt.
`tests/test_payment_approval.py`	13	Approval levels and the threshold boundary, stored approvals, duplicate block and release, shareholder rule, trusted account, company rule, access checks.
`tests/test_overrides.py`	11	For each overridden standard method: a negative, a normal and a standard-preserved test, plus the outstanding-account case.

Executed result: the full suite of the three RVK modules (74 tests) was executed by the delivery team on 2026-10-15 on a fresh database (`v19_rvk_docs`, installed with `--test-enable`): 0 failed, 0 errors. This module contributes 28 of those tests. This guide's author did not re-run them.

Fixed on 2026-10-15: approval bypass with an outstanding account

On a database whose payment method has an outstanding account, Odoo 19's Register Payment dialog generates the journal entry while creating the payment and marks the payment In Process without calling `action_post`. A vendor payment that needed approval therefore skipped the approval check. Fixed by overriding `account.payment._generate_journal_entry` (`models/account_payment.py:176`) to put waiting vendor payments back to draft with their entry unposted. Covered by `tests/test_overrides.py:101` (`test_o1_large_payment_waits_with_outstanding_account`), part of the passing run above. Observed on the documentation database the same day: registering a 60,000 SAR matched bill (payment outstanding account `101004 Outstanding Payments`) created the journal entry at registration, yet left the payment in Draft with status To Approve: Finance Manager, its entry in draft and the bill not paid.

Licence and authorship

Manifest: licence `Other proprietary`, author `OdooTec`, website `https://www.odootec.com`; no maintainer key. A `LICENSE` file is present (OdooTec Development Licence ODTL-1, version 1.4).

Compliance with the OdooTec module standard

Compliance with the OdooTec module standard was not checked: the standard reference file is empty.

Open points

Unfinished work

No TODO, FIXME or XXX comments were found. No stub methods, missing data files or unimported Python files were found by the static scan.

Behaviour gaps found while documenting

Finding	Evidence	Status
An approval stays valid when the amount is edited afterwards within the same band. A payment of 60,000 SAR approved by the Finance Manager, then edited to 199,999 SAR by the accountant, kept Approved and posted.	Reproduced in an odoo shell transaction on 2026-10-15, then rolled back. `_compute_approval_state` only compares the levels given with the levels required (`models/account_payment.py:105`).	Open; no test covers it. Consider clearing approvals when amount, partner or currency changes.
No segregation of duties: the user who creates a payment can approve it if they hold the level's group.	`action_approve` checks only the group (`models/account_payment.py:206`).	Open; by design per README ("one user holding several approver groups approves one level per click").
The duplicate key does not include the currency: same partner, amount and memo in two currencies count as duplicates.	Domain in `_find_duplicate` (`models/account_payment.py:354`).	Read in code, not tested.
A draft blocked as a duplicate is not recomputed when the earlier payment is later cancelled; `duplicate_of_id` depends only on the draft's own fields.	`@api.depends` on `_compute_duplicate_of_id` (`models/account_payment.py:122`).	Read in code, not tested.
Finance Manager visibility depends on the `accountant` app (see Prerequisites).	Checked with `get_views` and visible menus for a user holding only Accounting / Administrator.	Open; configuration or manifest decision.

README items listed as known limitations (no daily payment run, standing bank instructions or automatic payment of recurring bills) are out of scope, not defects.

Translations

`i18n/ar.po`: 89 entries, 0 untranslated, 0 fuzzy. The static scan reported 2 strings as missing (`This payment repeats` and the banner sentence in `views/account_payment_views.xml`); a manual check

found both in the catalogue as `<span>…</span>` terms (`i18n/ar.po:65` and `:71`), which is how Odoo exports view text, so nothing is actually missing. Extraction is static and approximate.

Reports

The module defines no QWeb report.

Documentation gaps

- Tests were not re-run by the author of this guide; the result cited is the delivery team's run of 2026-10-15.
- Batch payment refusal and the untrusted bank account refusal were read in code and tests only; they were not exercised on screen.
- Multi-company behaviour (record rule, thresholds per company) was not exercised on screen; the documentation database has one company.